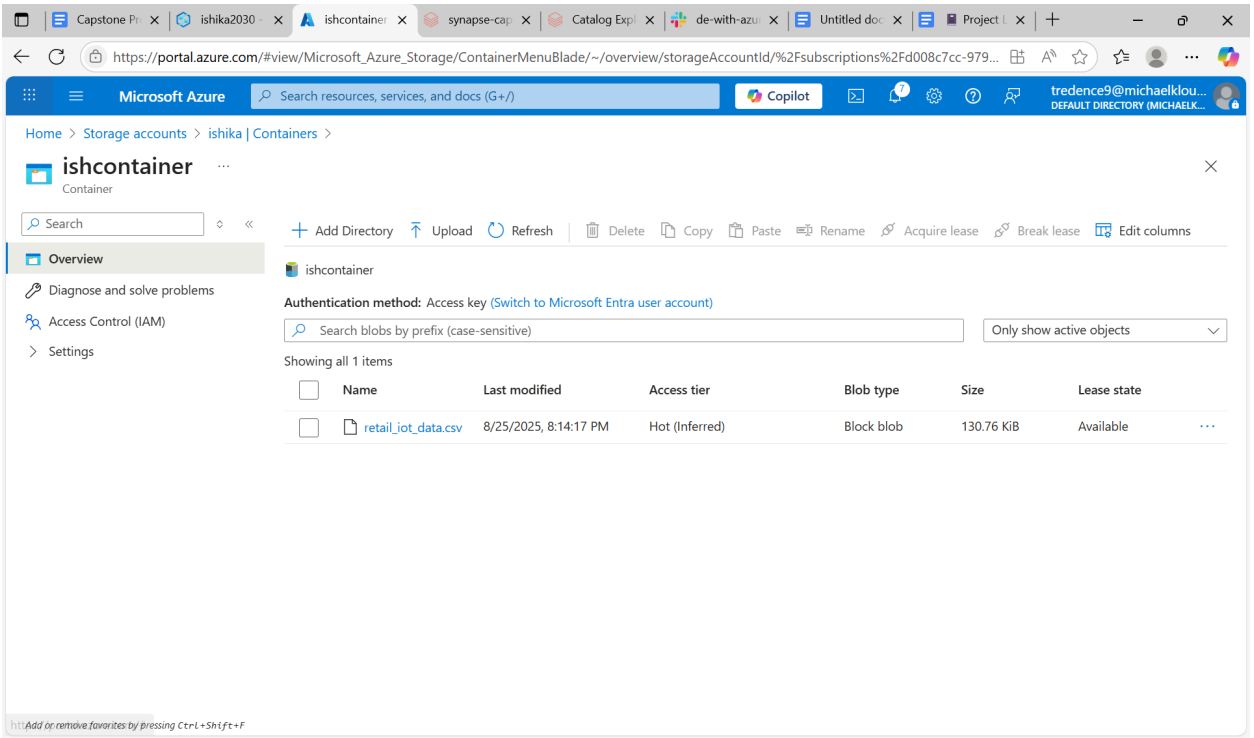
NAME - ISHIKA
EMP ID : 7145

Project Log : ETL Pipeline with Unity Catalog & DLT

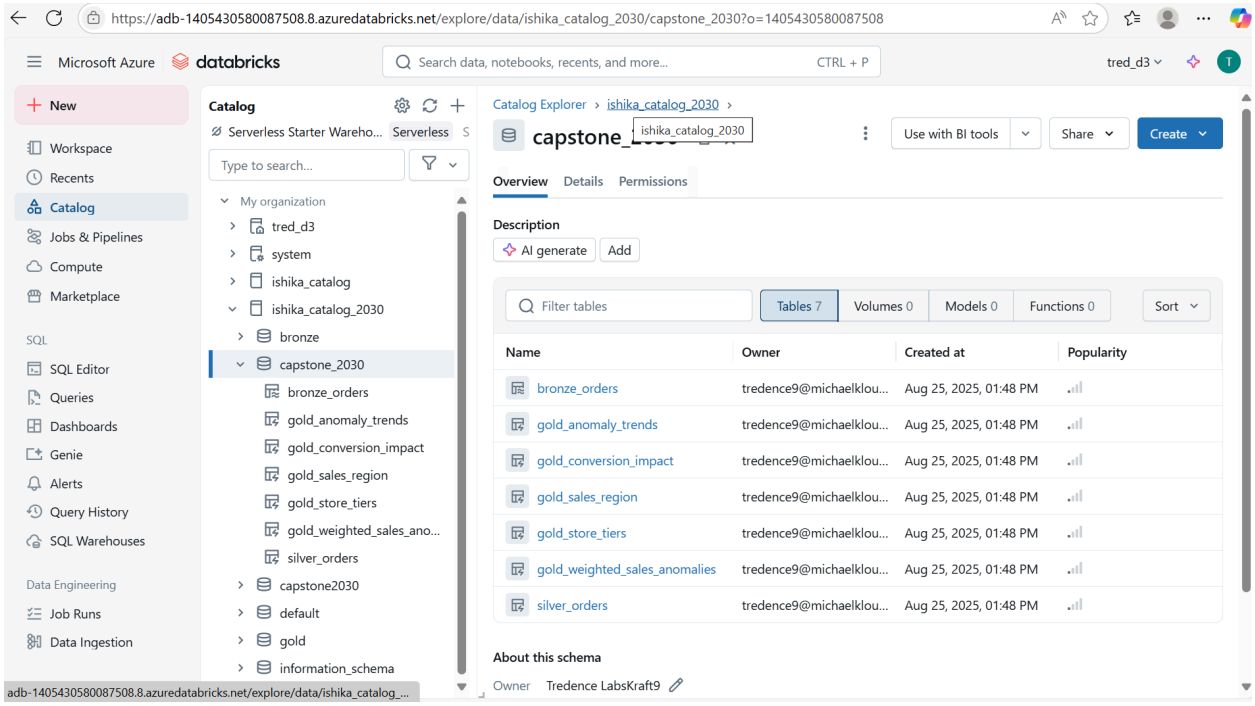
1. Azure Setup

- Created **ADLS Gen2 storage account**: `ishika`
- Created container:
    - `ishconatiner` → uploaded raw file `retail_iot_data.csv`



2. Unity Catalog Setup

- Created Unity Catalog: `ishika_catalog_2030`



- Created schema: capstone_2030

```
CREATE SCHEMA IF NOT EXISTS
ishika_catalog_2030.capstone_2030
COMMENT 'Schema for BI Integration demo with DLT';
```

3. Bronze Layer – Ingestion

- Created **DLT Notebook**: capstone-1
- Defined schema manually (to avoid Auto Loader empty path issue)

Used **Auto Loader** (`cloudFiles`) to stream data from ADLS

```python
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
IntegerType, TimestampType

# ✅ Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),       # can cast to Timestamp
later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

# ✅ Paths (use subdirectories to avoid overlap with DLT system files)
raw_path = "abfss://ishcontainer@ishika.dfs.core.windows.net/raw_data/"
schema_path =
"abfss://ishcontainer@ishika.dfs.core.windows.net/dlt_schemas/bronze_orders_het
ul"
checkpoint_path =
"abfss://ishcontainer@ishika.dfs.core.windows.net/dlt_checkpoints/bronze_orders
_hetul"

@table(
    name="ishika_catalog_2030.capstone_2030.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto Loader"
)
def bronze_orders():
```

```
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)    # ✅ manually set schema (no infer needed)
            .load(raw_path)        # ✅ points to /raw_data/ folder
    )
```

4. Silver Layer (Transformation)

Purpose: **Data cleaning, deduplication, anomaly handling, business rules**

✅ Implemented Silver transformation logic:

- Removed duplicates (`distinct`).
- Handled nulls with `fillna`.
- Standardized schema (ensured consistent datatypes).
- Joined device anomaly info with sales.
- Added **business rule**: flagged sales where anomaly occurred (`AnomalyFlag = 1`).

```
from dlt import table

import pyspark.sql.functions as F

from pyspark.sql.types import DoubleType, IntegerType



# ========================================

# Silver Layer - Cleaned IoT Orders

# ========================================
```

```
@table(
    name="capstone_2030.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("bronze_orders")

    # Standardize schema
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
              .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
              .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
              .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
              .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
    )

    # Deduplicate by OrderID
    orders = orders.dropDuplicates(["OrderID"])
```

```
    # Handle missing anomaly type
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) &
(F.col("AnomalyType").isNull()), "Unknown")
         .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") ==
""), "Unknown")
         .otherwise(F.col("AnomalyType"))
    )

    # Add business rule fields
    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
              .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
              .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
              .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )

    return orders
```

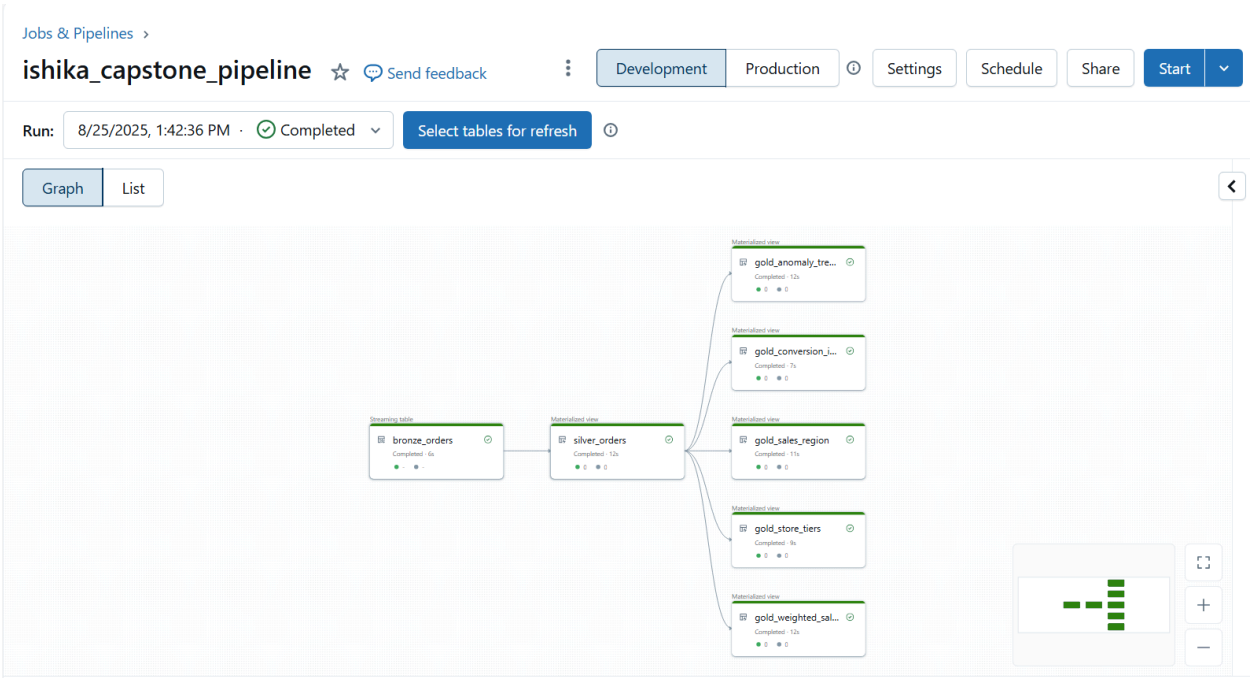
5. Gold Layer (Aggregation & Enrichment)

Purpose: **Business KPIs, aggregations, and reporting-ready data**

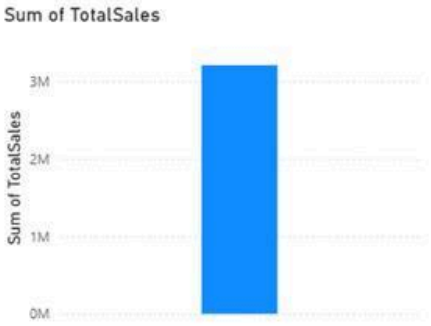
✅ Implemented **5 Gold tables**:

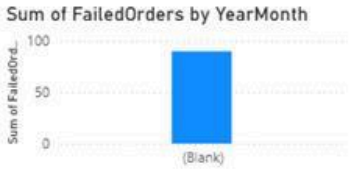
1. `gold_sales_region` → Daily/Monthly Sales per Region
2. `gold_anomaly_trends` → Device Anomaly Trend per Store
3. `gold_conversion_impact` → Conversion Rate Impact when devices fail
4. `gold_store_tiers` → Store classification (High/Medium/Low performers)
5. `gold_weighted_sales_anomalies` → Weighted average of Sales vs Anomalies

6. Create Pipeline : ishika_capstone_pipeline



Visualize in Power BI 👍





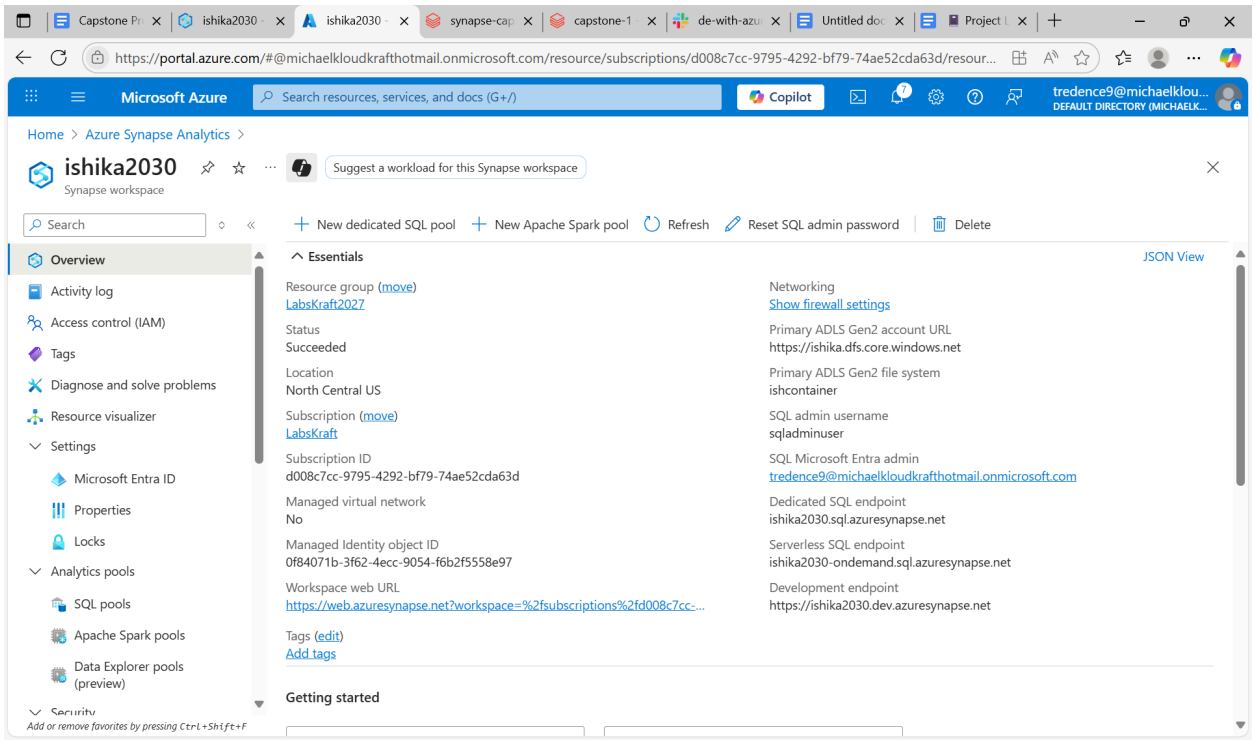
Loading Data from Different Sources

Capstone Retail Data Pipeline Report - Azure Synapse

1. Azure Synapse Workspace Setup

- **Action Taken: Created Azure Synapse Workspace**
- **Workspace Name: <ishika2030>**
- **Region: <Central-US>**
- **Subscription: <d008c7cc-9795-4292-bf79-74ae52cda63d>**

- **Resource Group: `<LabsKraft2027>`**
- **Expected Outcome: Workspace created successfully**
- **Actual Outcome: ✅ Workspace provisioned successfully**



---

2. SQL Dedicated Pool / Database Setup

- **Action Taken: Created SQL Dedicated Pool**
- **Database Name: `CapstoneRetailD`**
- **Performance Level: `<DW100c / DW200c>`**

- **Collation:** `SQL_Latin1_General_CP1_CI_AS`
- **Expected Outcome: SQL Pool ready to use**
- **Actual Outcome: ✅ Pool created and available**

---

3. Historical Sales Table Setup in Synapse

- **Action Taken: Created Historical Bronze Table**
- **Table Name:** `dbo.RetailIoTData`
- **Schema:**
    - `OrderID, CustomerID, Region, StoreID, Product, Quantity, UnitPrice, SalesAmount, OrderTime`

Validation Query:

```
SELECT TOP 10 * FROM dbo.RetailIoTData;
```

---

4. Data Pipeline Execution (Bronze → Silver → Gold)

- **Pipeline Name: ishika_capstone_etl**
- **Run Mode:** `<Continuous / Triggered>`

📊 **Data Ingestion**

- From CSV Files: `<row count>`
- From Event Hub: `<row count>`
- From Synapse Historical Sales: `<row count>`
- Unified Bronze Total: `<row count>`

🔄 Silver Transformations

- Deduplication applied
- Null handling on `OrderID`, `DeviceType`, `AnomalyType` (replaced with `"None"`)
- Type casting of numeric & datetime columns
- Removal of invalid rows

✨ Gold Transformations

- Aggregated KPIs:
  - Daily & Monthly Sales per Region
  - Device Anomaly Trends per Store
  - Conversion Rate Impact when devices fail
- Business Enrichment:
  - Store Tier Classification (High/Medium/Low performers)
  - Weighted Average of Sales vs. Anomalies
- Optimization:

- Partitioned by `Region, OrderDate`
- Z-Ordering on `ProductID, StoreID`
- Compaction with `OPTIMIZE`
- Storage cleanup with `VACUUM`

---

5. Verification Queries

Bronze Layer Check

```
SELECT TOP 10 *
FROM ishika_catalog_2030.capstone_2030.bronze_orders;
```

Silver Layer Check

```
SELECT TOP 10 *
FROM ishika_catalog_2030.capstone_2030.silver_orders;
```

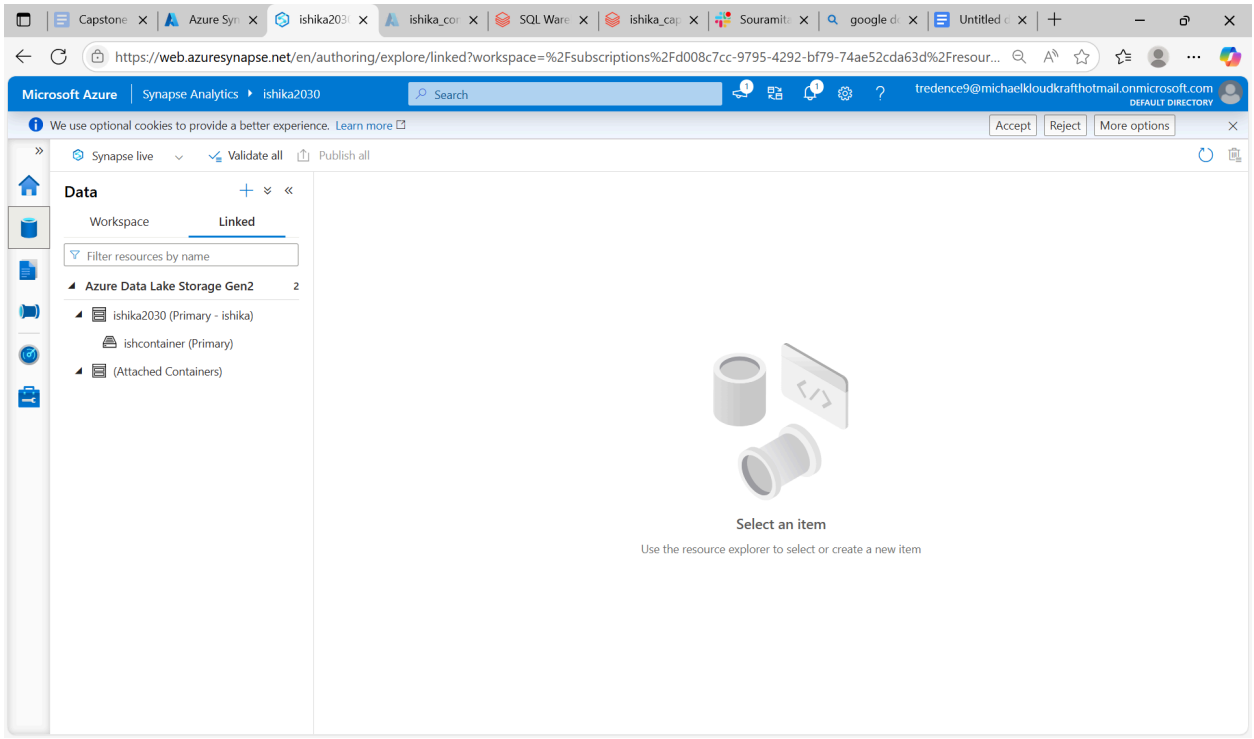
Gold – Sales by Region

```
SELECT TOP 10 *
FROM ishika_catalog_2030.capstone_2030.gold_sales_region;
```

Gold – Anomaly Trends

```
SELECT TOP 10 *
FROM ishika_catalog_2030.capstone_2030.gold_anomaly_trends;
```

- **Verification Logs / Output:**

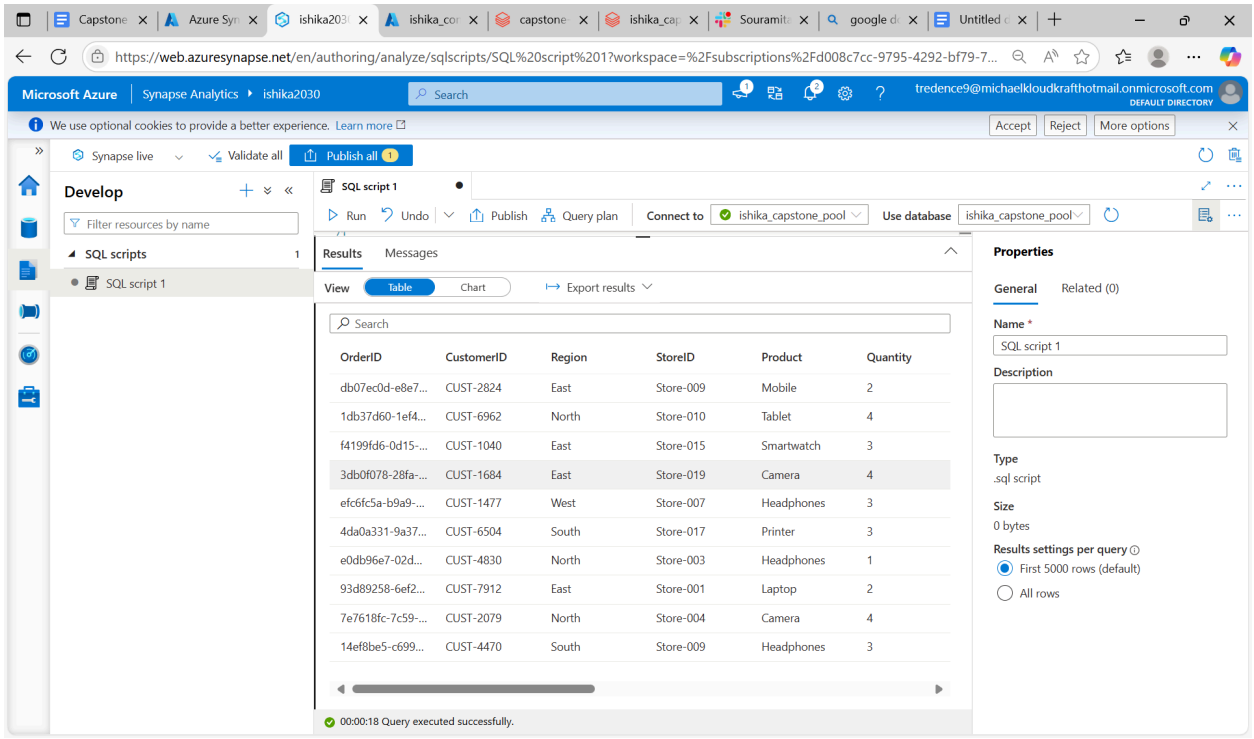


Write the SQL query in NEW SQL Script in Develop section-

```
-- ============================
-- 0️⃣ Drop existing objects if they exist
-- ============================

IF OBJECT_ID('dbo.RetailIoTData', 'U') IS NOT NULL
    DROP TABLE dbo.RetailIoTData;

IF OBJECT_ID('ext_RetailIoTData', 'U') IS NOT NULL
    DROP EXTERNAL TABLE ext_RetailIoTData;

IF EXISTS (SELECT * FROM sys.external_file_formats WHERE name =
'CsvFormat')
    DROP EXTERNAL FILE FORMAT CsvFormat;

IF EXISTS (SELECT * FROM sys.external_data_sources WHERE name = 'IoTADLS')
    DROP EXTERNAL DATA SOURCE IoTADLS;

-- ============================
```

```sql
-- 1️⃣ Create external file format for CSV
-- ============================

CREATE EXTERNAL FILE FORMAT CsvFormat
WITH (
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS (
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        FIRST_ROW = 2,
        USE_TYPE_DEFAULT = TRUE
    )
);

-- ============================
-- 2️⃣ Create external data source pointing to the folder in ADLS
-- ============================

CREATE EXTERNAL DATA SOURCE IoTADLS
WITH (
    LOCATION =
'abfss://ishcontainer@ishika.dfs.core.windows.net/incoming2030/',
    TYPE = HADOOP
);

-- ============================
-- 3️⃣ Create external table for IoT data
-- ============================

CREATE EXTERNAL TABLE ext_RetailIoTData (
    OrderID NVARCHAR(50),
    CustomerID NVARCHAR(50),
    Region NVARCHAR(50),
    StoreID NVARCHAR(50),
    Product NVARCHAR(100),
    Quantity INT,
    UnitPrice FLOAT,
    SalesAmount FLOAT,
    OrderTime DATETIME,
    DeviceType NVARCHAR(50),
```

```
    Temperature FLOAT,
    DeviceStatus NVARCHAR(50),
    AnomalyFlag INT,
    AnomalyType NVARCHAR(50)
)
WITH (
    LOCATION = 'retail_iot_data.csv',   -- file inside raw_data folder
    DATA_SOURCE = IoTADLS,
    FILE_FORMAT = CsvFormat
);

-- ============================
-- 4️⃣ Create managed table and load data
-- ============================

CREATE TABLE dbo.RetailIoTData
WITH
(
    DISTRIBUTION = ROUND_ROBIN,
    HEAP
)
AS
SELECT * FROM ext_RetailIoTData;

-- ============================
-- 5️⃣ Verify the table
-- ============================

SELECT TOP 10 * FROM dbo.RetailIoTData;
```



Synapse code:

```
%python
# Databricks Delta Live Tables (DLT) - Bronze Layer
# Catalog : ishika_catalog_2030
# Schema  : capstone_2030
# Database: ishikaRetailDB
# then create a DLT pipeline that points to this notebook.

import dlt
import pyspark.sql.functions as F
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType, IntegerType, TimestampType
)

# ---------------------------------------------------------------------
# Configuration / secrets (DO NOT hardcode secrets in production)
# Create a Databricks Secret Scope (e.g. "capstone-scope") with keys:
#  - synapse-username
#  - synapse-password
# ---------------------------------------------------------------------
jdbc_hostname = "ishika2030.sql.azuresynapse.net"
jdbc_port     = "1433"
jdbc_database = "ishikaRetailDB"   # as requested
jdbc_url      = f"jdbc:sqlserver://{jdbc_hostname}:{jdbc_port};database={jdbc_database};encrypt=true;
trustServerCertificate=false;loginTimeout=30;"
```

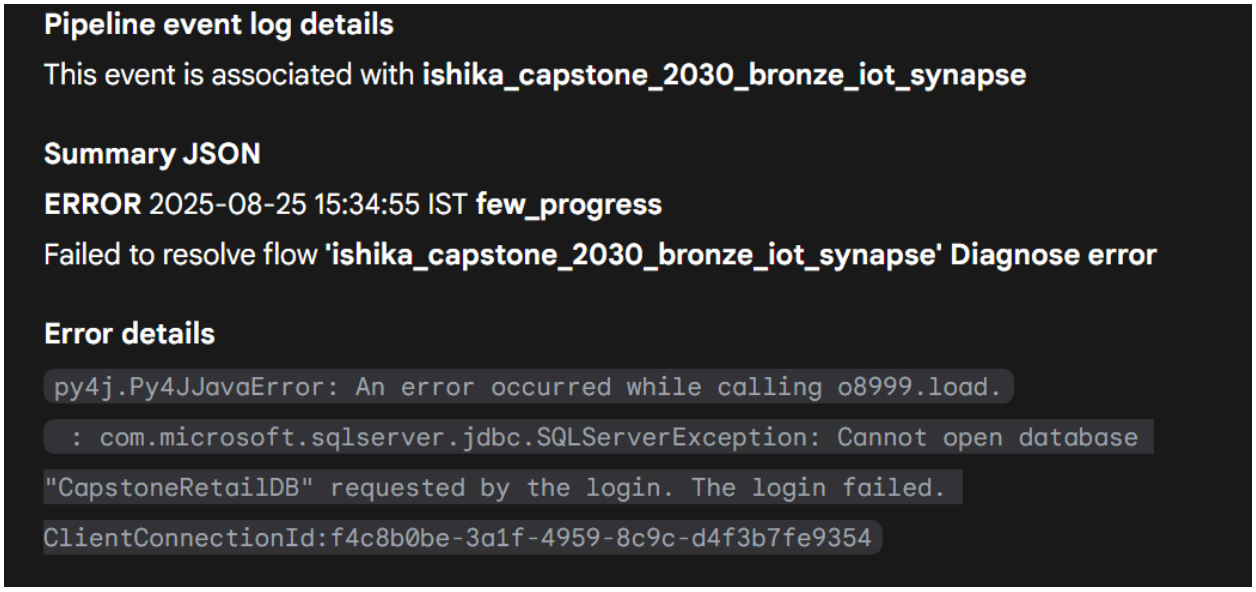
```python
# retrieve credentials from Databricks secrets (replace scope/key names if different)
jdbc_username = dbutils.secrets.get("capstone-scope", "synapse-username")
jdbc_password = dbutils.secrets.get("capstone-scope", "synapse-password")

# JDBC table names in Synapse (adjust if different)
historical_table = "dbo.HistoricalSales"
iot_table        = "dbo.RetailIoTData"

# ADLS / Auto Loader paths (adjust to your storage/account)
raw_path     = "abfss://ishcontainer@ishika.dfs.core.windows.net/incoming2030/"
schema_path  = "abfss://ishcontainer@ishika.dfs.core.windows.net/dlt_schemas/bronze_orders/"
checkpoint_path = "abfss://ishcontainer@ishika.dfs.core.windows.net/dlt_checkpoints/bronze_orders/"

# ---------------------------
# 1) Bronze - IoT CSV (Auto Loader -> DLT table)
# ---------------------------
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
```

```python
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),       # parse later to timestamp
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

@dlt.table(
    name="ishika_catalog_2030.capstone_2030.bronze_orders",
    comment="Bronze table - raw IoT CSV ingested via Auto Loader"
)
def bronze_orders():
    # Use Auto Loader (cloudFiles) to continuously ingest files placed in `raw_path`
    df = (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", "true")
            .option("cloudFiles.schemaLocation", schema_path)
            .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
            .schema(iot_schema)
            .load(raw_path)
    )
```

```
    # Standardize and cast OrderTime to timestamp (adjust format if necessary)
    df = df.withColumn("OrderTime", F.to_timestamp("OrderTime", "yyyy-MM-dd HH:mm:ss"))

    # Return raw stream to DLT bronze_orders table (DLT will manage lineage/checkpoints)
    return df

# ---------------------------
# 2) Bronze - Historical Sales (Synapse JDBC read)
# ---------------------------
@dlt.table(
    name="ishika_catalog_2030.capstone_2030.bronze_historical_sales",
    comment="Bronze table - historical sales loaded from Azure Synapse (JDBC)"
)
def bronze_historical_sales():
    # Read historical sales via JDBC. If the read fails, DLT will surface the error in pipeline logs.
    return (
        spark.read.format("jdbc")
            .option("url", jdbc_url)
            .option("dbtable", historical_table)
            .option("user", jdbc_username)
            .option("password", jdbc_password)
            .option("driver", "com.microsoft.sqlserver.jdbc.SQLServerDriver")
            .load()
            # Ensure OrderTime is a timestamp type (Synapse may already have it as datetime)
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "yyyy-MM-dd HH:mm:ss"))
```

```
# ---------------------------
# 3) Bronze - IoT data coming from Synapse (RetailIoTData)
#     (This creates the DLT table name you specified)
# ---------------------------
@dlt.table(
    name="ishika_catalog_2030.capstone_2030.bronze_iot_synapse",
    comment="Bronze table - IoT retail data read from Azure Synapse (RetailIoTData)"
)
def bronze_iot_synapse():
    df = (
        spark.read.format("jdbc")
            .option("url", jdbc_url)
            .option("dbtable", iot_table)
            .option("user", jdbc_username)
            .option("password", jdbc_password)
            .option("driver", "com.microsoft.sqlserver.jdbc.SQLServerDriver")
            .load()
    )
    # Normalize the timestamp column name & type if needed
    df = df.withColumn("OrderTime", F.to_timestamp("OrderTime", "yyyy-MM-dd HH:mm:ss"))
    return df

# ---------------------------
# Notes / Best practices
#  - Replace secret scope/key names with the ones you created.
#  - Ensure the cluster attached to the DLT pipeline has the JDBC driver library:
```

Pipeline Error:



EVENTHUB :

1️⃣ Virtual Machine Setup (Data Generation)

- **Task:** Generate synthetic IoT data every second.

Schema:

`OrderID, CustomerID, Region, StoreID, Product, Quantity, UnitPrice, SalesAmount, OrderTime, DeviceType, Temperature, DeviceStatus, AnomalyFlag, AnomalyType`

-
- **Python Script:** `iot_stream.py`
    - Generates 1 row per second.

- Writes CSV locally.

Example row generation logged:

```
Added row: {'OrderID': 'uuid', 'CustomerID': 'CUST1234', ... }
```

-

2️⃣ Event Hub Setup

- **Event Space:** ishika-eventhub
- **Event Hub:** `ishika-event-capstone`

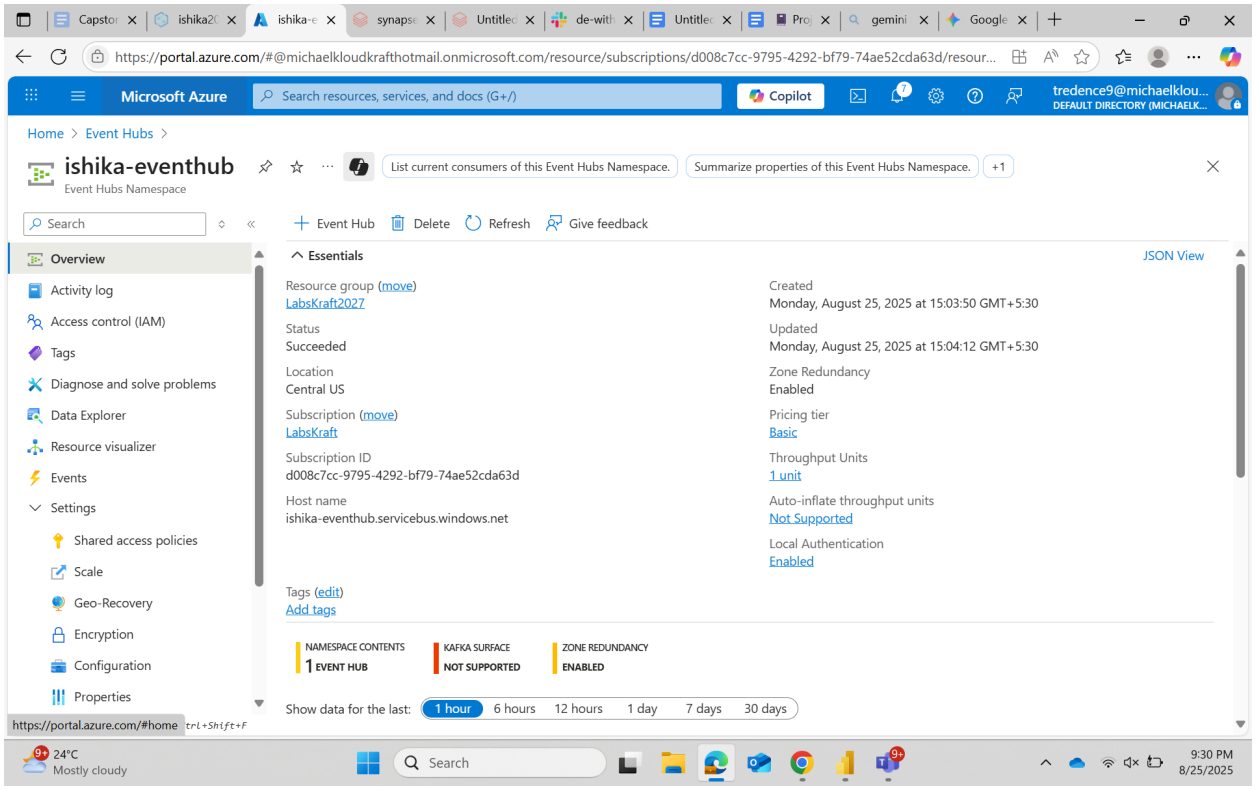
Connection String Example:

```
Endpoint=sb://ishika-eventhub.servicebus.windows.net/;
SharedAccessKeyName=hello;
SharedAccessKey=<key>;
EntityPath=hetulhub
```

- **VM:** Script updated to send rows to Event Hub using `azure-eventhub` SDK.

Log Example:

```
Sent event to Event Hub: {'OrderID': '...', 'CustomerID': '...', ...}
```



3 Databricks DLT Notebook (Bronze Layer)

- **Bronze tables:**
    1. `bronze_orders` – from CSV (Auto Loader).
    2. `bronze_iot_synapse` – from Synapse table.
-

---

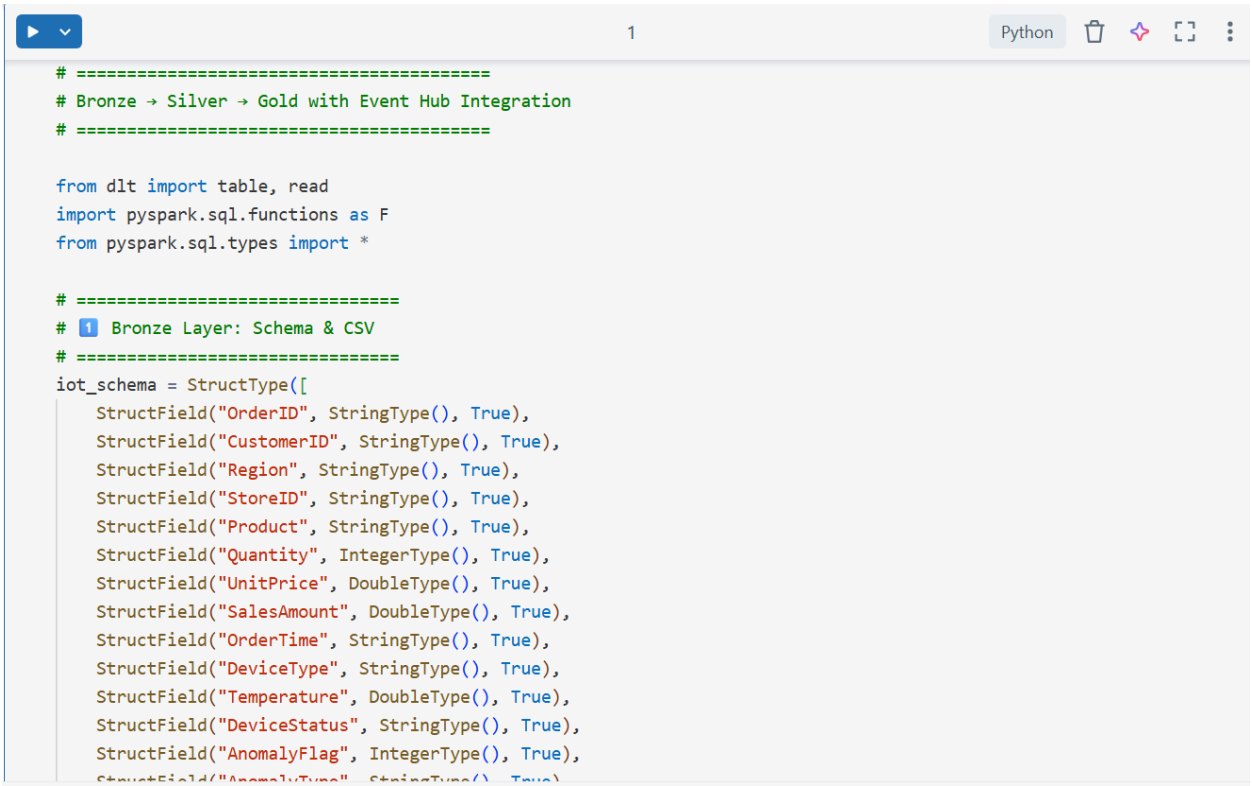
4 Silver Layer (Cleaning + Transformation)

- Deduplicated `OrderID`.
- Cast types for Quantity, UnitPrice, SalesAmount, Temperature, OrderTime.

- Handle missing `AnomalyType`.
- Add derived columns: `IsAnomaly`, `NormalizedStoreID`, `NormalizedProduct`.

5️⃣ Gold Layer (Aggregations & Business Metrics)

- **Tables:**
  - `gold_sales_region` – daily/monthly sales per region
  - `gold_anomaly_trends` – anomaly trends per store
  - `gold_conversion_impact` – failure rate per store
  - `gold_store_tiers` – tier classification based on sales
  - `gold_weighted_sales_anomalies` – sales per anomaly



```python
raw_path = "abfss://ishcontainer@ishika.dfs.core.windows.net/raw_data/"
schema_path = "abfss://ishcontainer@ishika.dfs.core.windows.net/dlt_schemas/bronze_orders_ishika"

@table(
    name="ishika_catalog_2030.capstone_2030.bronze_orders_csv",
    comment="Bronze table - raw IoT CSV"
)
def bronze_orders_csv():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)
            .load(raw_path)
            .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    )

# ================================
# 2️⃣ Bronze Layer: Event Hub IoT
# ================================
eventhub_connection_string = "Endpoint=sb://<your-eventhub-namespace>.servicebus.windows.net/;
SharedAccessKeyName=<your-policy>;SharedAccessKey=<your-key>"
consumer_group = "$Default"
```

```python
ehConf = {
    'eventhubs.connectionString': eventhub_connection_string,
    'eventhubs.consumerGroup': consumer_group,
    'eventhubs.startingPosition': '@latest'
}

@table(
    name="ishika_catalog_2030.capstone_2030.bronze_orders_eventhub",
    comment="Bronze table - live IoT data from Event Hub"
)
def bronze_orders_eventhub():
    df = spark.readStream \
        .format("eventhubs") \
        .options(**ehConf) \
        .load()
    df1 = df.selectExpr("cast(body as string) as json_string")
    df_parsed = df1.select(F.from_json(F.col("json_string"), iot_schema).alias("data")).select("data.*")
    return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))

# ================================
# 3️⃣ Unified Bronze Table
# ================================
@table(
    name="ishika_catalog_2030.capstone_2030.bronze_orders",
    comment="Unified Bronze table (CSV + Event Hub)"
```

```
)
def bronze_orders():
    df_csv = read("ishika_catalog_2030.capstone_2030.bronze_orders_csv")
    df_eh = read("ishika_catalog_2030.capstone_2030.bronze_orders_eventhub")
    return df_csv.unionByName(df_eh)

# ================================
# 4️⃣ Silver Layer
# ================================
@table(
    name="ishika_catalog_2030.capstone_2030.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    orders = read("ishika_catalog_2030.capstone_2030.bronze_orders")

    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
              .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
              .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
              .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
              .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
    )

    orders = orders.dropDuplicates(["OrderID"])
```

```
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
         .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
         .otherwise(F.col("AnomalyType"))
    )

    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
              .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
              .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
              .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )

    return orders

# ================================
# 5️⃣ Gold Layer
# ================================
@table(
    name="ishika_catalog_2030.capstone_2030.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
```

```
def gold_sales_region():
    return (
        read("ishika_catalog_2030.capstone_2030.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

@table(
    name="ishika_catalog_2030.capstone_2030.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        read("ishika_catalog_2030.capstone_2030.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )

@table(
    name="ishika_catalog_2030.capstone_2030.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)
```

```
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )

@table(
    name="ishika_catalog_2030.capstone_2030.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        read("ishika_catalog_2030.capstone_2030.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )
```

```
        )
    )

@table(
    name="ishika_catalog_2030.capstone_2030.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = read("ishika_catalog_2030.capstone_2030.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
             .otherwise(F.lit(None))
        )
    )
```

Pipeline Error:

Pipeline event log details

This event is associated with **ishika_capstone_2030_bronze_orders_eventhub**

Summary JSON

ERROR 2025-08-25 16:34:08 IST **few_progress**

Failed to resolve flow: **ishika_capstone_2030_bronze_orders_eventhub Diagnose error**

Error details

```
pyspark.errors.exceptions.captured.AnalysisException: Traceback (most recent call last):
  File "/Users/trudence/d@ishikakloudcraft@hetul.onmicrosoft.com/eventhub_project01/_call_5.line_68_in_bronze_orders_eventhub
...load())

pyspark.errors.exceptions.captured.AnalysisException: [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhub is not
```